MATERIA

INF560 — Desarrollo Web Backend

GUÍA DE LABORATORIO 4

Configuración del Proyecto y Base de Datos

Material de Estudio Teórico-Práctico

CONTENIDO DE LA GUÍA

1. Creación del proyecto Laravel 13
2. Configuración de PostgreSQL
3. Diseño del esquema de base de datos
4. Migraciones para authors, categories, books
5. Tabla pivote author_book
6. Migraciones para members y loans
7. Versionamiento con Git y GitHub

COMPETENCIA DE LA GUÍA

Configurar un proyecto Laravel 13 conectado a PostgreSQL, diseñar e implementar el esquema de base de datos de una biblioteca mediante migraciones, y gestionar el versionamiento del código fuente con Git y GitHub.

1. Objetivos de la Guía

Al finalizar esta guía, el estudiante será capaz de:

- Crear un proyecto Laravel 13 desde cero y comprender su estructura de directorios.
- Configurar la conexión a una base de datos PostgreSQL mediante el archivo .env.
- Diseñar un esquema de base de datos relacional para un sistema de biblioteca.
- Implementar migraciones con claves foráneas, índices y restricciones.
- Utilizar Git para versionar el proyecto y publicarlo en GitHub.

2. Prerrequisitos

Antes de comenzar esta guía, asegúrese de contar con:

- **PHP 8.3+** instalado y funcionando (verificar con php -v).
- **Composer** instalado globalmente (verificar con composer --version).
- **PostgreSQL 15+** instalado y en ejecución (verificar con psql --version).
- **Git** instalado (verificar con git --version).
- **Cuenta en GitHub** creada.
- **Editor de código**: VS Code recomendado.

> **Nota importante**
> Esta guía asume que ya completó la Guía 3 de configuración del entorno de desarrollo. Si aún no tiene Laravel y PostgreSQL instalados, consulte dicha guía antes de continuar.

3. Desarrollo Paso a Paso

3.1 Crear el Proyecto Laravel

Abra su terminal y ejecute el siguiente comando para crear un nuevo proyecto Laravel 13:

```
composer create-project laravel/laravel library-app
```

Este comando descarga Laravel 13 con todas sus dependencias. Una vez finalizado, navegue al directorio del proyecto:

```
cd library-app
```

3.1.1 Estructura de Directorios

Laravel organiza su código en una estructura clara y predecible. Explore las carpetas principales:

Directorio	Descripción
`app/`	Contiene el código principal: Models, Http/Controllers, Providers. Aquí vive la lógica de negocio.
`bootstrap/`	Archivo app.php que inicializa el framework. Punto de entrada de la aplicación.
`config/`	Archivos de configuración (database.php, app.php, mail.php, etc.).

Directorio	Descripción
`database/`	Migraciones, seeders y factories para la base de datos.
`resources/views/`	Plantillas Blade (.blade.php) para las vistas HTML.
`routes/`	Definición de rutas: web.php (rutas web) y api.php (rutas API).
`storage/`	Archivos generados: logs, caché, archivos subidos.
`public/`	Punto de entrada web (index.php), assets públicos (CSS, JS, imágenes).
`.env`	Variables de entorno (credenciales, configuración local). NO se versiona.

Definición: Archivo .env

El archivo .env contiene variables de entorno específicas de cada instalación. Incluye credenciales de base de datos, claves de API y configuraciones que varían entre desarrollo y producción. Laravel carga estas variables automáticamente mediante la librería Dotenv.

3.1.2 Configurar la Conexión a PostgreSQL

Abra el archivo **.env** en la raíz del proyecto y modifique las siguientes líneas:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=library_app
DB_USERNAME=postgres
DB_PASSWORD=su_contraseña_aqui
```

Advertencia

Reemplace su_contraseña_aqui con la contraseña real de su usuario PostgreSQL. Nunca comparta ni publique este archivo con contraseñas reales.

3.1.3 Crear Usuario, Base de Datos y Asignar Privilegios

Este es el flujo completo desde cero: primero se crea el usuario, luego la base de datos y finalmente se le asignan todos los permisos necesarios

```
-- ────────────────────────────────────────────
-- PASO 1: Conectarse como superusuario
-- ────────────────────────────────────────────
psql -U postgres

-- ────────────────────────────────────────────
-- PASO 2: Crear el usuario con contraseña
-- ────────────────────────────────────────────
CREATE USER library_user WITH PASSWORD 'contraseña_segura';

-- Verificar que el usuario fue creado
\du

-- ────────────────────────────────────────────
-- PASO 3: Crear la base de datos con ese usuario como dueño
-- ────────────────────────────────────────────
CREATE DATABASE library_db OWNER library_user;
```

```
-- ──────────────────────────────────────────────
-- PASO 4: Dar todos los privilegios sobre la BD
-- ──────────────────────────────────────────────
GRANT ALL PRIVILEGES ON DATABASE library_db TO library_user;

-- Verificar que la BD fue creada con el owner correcto
\l

-- ──────────────────────────────────────────────
-- PASO 5: Conectarse a la BD y ajustar el esquema (PostgreSQL 15+)
-- ──────────────────────────────────────────────
\c library_db

GRANT ALL ON SCHEMA public TO library_user;

-- ──────────────────────────────────────────────
-- PASO 6: Salir
-- ──────────────────────────────────────────────
\q

-- ──────────────────────────────────────────────
-- VERIFICACIÓN: Conectarse con el nuevo usuario
-- ──────────────────────────────────────────────
psql -U library_user -d library_db
```

3.1.4 Verificar la Conexión

Regrese a la terminal del proyecto y ejecute:

```
php artisan db:show
```

Si la conexión es exitosa, verá información sobre la base de datos: nombre, driver (pgsql), host, puerto y estado de las tablas. Si obtiene un error, verifique las credenciales en el archivo .env y que PostgreSQL esté en ejecución.

Ejemplo de salida esperada

El comando db:show mostrará algo similar a:

```
PostgreSQL ........................................ pgsql
Database ................................. library_db
Host ..................................... 127.0.0.1
Port ............................................ 5432
Username ..................................... postgres
Open Connections ................................... 1
Tables ............................................. 0
```

3.2 Diseño del Esquema de Base de Datos

El sistema de biblioteca gestiona las siguientes entidades principales: **autores**, **categorías**, **libros**, **miembros** y **préstamos**. A continuación se presenta el diagrama de relaciones entre las tablas:

Diagrama de Relaciones

users 1 —— 1 members (un usuario puede ser un miembro)
members 1 —— N loans (un miembro puede tener múltiples préstamos)

```
books       1 ——— N  loans      (un libro puede estar en múltiples préstamos)
categories  1 ——— N  books      (una categoría tiene múltiples libros)
authors     N ——— N  books      (relación muchos a muchos via author_book)
users       1 ——— N  loans      (un usuario/bibliotecario registra préstamos)
```

A continuación se detalla la estructura de cada tabla. Cada tabla incluye campos de auditoría (timestamps) para rastrear cuándo se crearon y modificaron los registros.

Tabla: authors

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment
`first_name`	string(100)	Requerido
`last_name`	string(100)	Requerido
`nationality`	string(80)	Nullable
`birth_date`	date	Nullable
`biography`	text	Nullable
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Tabla: categories

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment
`name`	string(100)	Requerido, unique
`slug`	string(120)	Requerido, unique
`description`	text	Nullable
`color`	string(7)	Default: '#6B7280'
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Tabla: books

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment
`title`	string(255)	Requerido
`isbn`	string(13)	Requerido, unique
`publisher`	string(200)	Nullable
`publish_year`	unsignedSmallInteger	Nullable
`pages`	unsignedInteger	Nullable
`language`	string(50)	Default: 'Español'
`description`	text	Nullable
`cover_url`	string(500)	Nullable
`total_copies`	unsignedInteger	Default: 1
`available_copies`	unsignedInteger	Default: 1

Campo	Tipo	Restricciones
`status`	enum	available, reserved, unavailable. Default: 'available'
`category_id`	FK → categories	Nullable, SET NULL on delete
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto
`deleted_at`	timestamp	Nullable (SoftDeletes)

> **Nota: SoftDeletes**
> La tabla books utiliza SoftDeletes, lo que significa que al eliminar un libro, no se borra físicamente de la base de datos. En su lugar, se establece una fecha en el campo deleted_at. Esto permite recuperar libros eliminados y mantener la integridad referencial con los préstamos históricos.

Tabla: author_book (pivote)

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment
`author_id`	FK → authors	CASCADE on delete
`book_id`	FK → books	CASCADE on delete
`role`	string(50)	Default: 'Autor' (Autor, Coautor, Editor)
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Restricción adicional: índice único compuesto en [author_id, book_id] para evitar duplicados.

Tabla: members

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment
`user_id`	FK → users	CASCADE on delete, unique
`member_code`	string(20)	Unique. Formato: LIB-YYYYMMDD-XXXX
`phone`	string(20)	Nullable
`address`	text	Nullable
`membership_type`	enum	standard, premium, student. Default: 'standard'
`membership_expires_at`	date	Nullable
`max_loans`	unsignedTinyInteger	Default: 3
`is_active`	boolean	Default: true
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Tabla: loans

Campo	Tipo	Restricciones
`id`	bigIncrements	PK, auto-increment

Campo	Tipo	Restricciones
`book_id`	FK → books	RESTRICT on delete
`member_id`	FK → members	RESTRICT on delete
`loaned_by`	FK → users	Nullable, SET NULL on delete
`loan_date`	date	Requerido
`due_date`	date	Requerido
`returned_date`	date	Nullable (null = no devuelto)
`status`	enum	active, returned, overdue. Default: 'active'
`notes`	text	Nullable
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Comportamiento onDelete

CASCADE: al eliminar el registro padre, se eliminan automáticamente los registros hijos. Se usa en author_book y members.

SET NULL: al eliminar el registro padre, el campo FK se establece en NULL. Se usa en books.category_id y loans.loaned_by.

RESTRICT: impide eliminar el registro padre si existen registros hijos. Se usa en loans para proteger la integridad de los préstamos activos.

3.3 Crear las Migraciones

Laravel utiliza migraciones para definir la estructura de la base de datos de forma versionada y reproducible. Crearemos las migraciones en el orden correcto para respetar las dependencias entre tablas.

3.3.1 Verificar la Migración de Users

Laravel ya incluye una migración para la tabla users. Verifique que existe en **database/migrations/** con un nombre similar a 0001_01_01_000000_create_users_table.php. Esta migración ya contiene los campos id, name, email y password que necesitamos.

3.3.2 Generar las Migraciones

Ejecute los siguientes comandos en orden:

```
php artisan make:migration create_authors_table
php artisan make:migration create_categories_table
php artisan make:migration create_books_table
php artisan make:migration create_author_book_table
php artisan make:migration create_members_table
php artisan make:migration create_loans_table
```

Cada comando crea un archivo en **database/migrations/** con un timestamp único. A continuación, implemente el contenido de cada migración.

3.3.3 Migración: create_authors_table

Abra el archivo generado y reemplace su contenido con:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('authors', function (Blueprint $table) {
            $table->id();
            $table->string('first_name', 100);
            $table->string('last_name', 100);
            $table->string('nationality', 80)->nullable();
            $table->date('birth_date')->nullable();
            $table->text('biography')->nullable();
            $table->timestamps();

            // Índice para búsquedas frecuentes por nombre
            $table->index(['last_name', 'first_name']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('authors');
    }
};
```

3.3.4 Migración: create_categories_table

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->string('name', 100)->unique();
            $table->string('slug', 120)->unique();
            $table->text('description')->nullable();
            $table->string('color', 7)->default('#6B7280');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('categories');
    }
```

```
};
```

Definición: Slug

Un slug es una versión legible de un nombre, usada comúnmente en URLs. Por ejemplo, la categoría 'Ciencia Ficción' tendría el slug 'ciencia-ficcion'. Se genera reemplazando espacios por guiones y eliminando caracteres especiales.

3.3.5 Migración: create_books_table

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('books', function (Blueprint $table) {
            $table->id();
            $table->string('title', 255);
            $table->string('isbn', 13)->unique();
            $table->string('publisher', 200)->nullable();
            $table->unsignedSmallInteger('publish_year')->nullable();
            $table->unsignedInteger('pages')->nullable();
            $table->string('language', 50)->default('Español');
            $table->text('description')->nullable();
            $table->string('cover_url', 500)->nullable();
            $table->unsignedInteger('total_copies')->default(1);
            $table->unsignedInteger('available_copies')->default(1);
            $table->enum('status', ['available', 'reserved', 'unavailable'])
                  ->default('available');

            // FK a categories con SET NULL
            $table->foreignId('category_id')
                  ->nullable()
                  ->constrained('categories')
                  ->nullOnDelete();

            $table->timestamps();
            $table->softDeletes();

            // Índices para búsquedas comunes
            $table->index('title');
            $table->index('status');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('books');
    }
};
```

Nota: foreignId y constrained

El método foreignId('category_id') crea una columna unsignedBigInteger y, combinado con constrained('categories'), genera automáticamente la restricción de clave foránea apuntando a categories.id. El método nullOnDelete() configura el comportamiento SET NULL cuando se elimina la categoría padre.

3.3.6 Migración: create_author_book_table

Esta es la tabla pivote que resuelve la relación muchos a muchos entre autores y libros:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('author_book', function (Blueprint $table) {
            $table->id();

            $table->foreignId('author_id')
                  ->constrained('authors')
                  ->cascadeOnDelete();

            $table->foreignId('book_id')
                  ->constrained('books')
                  ->cascadeOnDelete();

            $table->string('role', 50)->default('Autor');
            $table->timestamps();

            // Evitar duplicados: un autor no puede estar
            // vinculado dos veces al mismo libro
            $table->unique(['author_id', 'book_id']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('author_book');
    }
};
```

Definición: Tabla Pivote

Una tabla pivote es una tabla intermedia que resuelve relaciones muchos a muchos (N:N). En este caso, un autor puede escribir muchos libros y un libro puede tener muchos autores. La tabla author_book almacena estas asociaciones junto con información adicional como el rol (Autor, Coautor, Editor).

3.3.7 Migración: create_members_table

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('members', function (Blueprint $table) {
            $table->id();

            $table->foreignId('user_id')
                  ->unique()
                  ->constrained('users')
                  ->cascadeOnDelete();

            $table->string('member_code', 20)->unique();
            $table->string('phone', 20)->nullable();
            $table->text('address')->nullable();
            $table->enum('membership_type', ['standard', 'premium', 'student'])
                  ->default('standard');
            $table->date('membership_expires_at')->nullable();
            $table->unsignedTinyInteger('max_loans')->default(3);
            $table->boolean('is_active')->default(true);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('members');
    }
};
```

> **Ejemplo: Código de Miembro**
> El campo member_code sigue el formato LIB-YYYYMMDD-XXXX, donde:
> LIB — Prefijo fijo que identifica al sistema de biblioteca
> YYYYMMDD — Fecha de registro del miembro
> XXXX — Número secuencial de 4 dígitos
> Ejemplo: LIB-20250315-0042

3.3.8 Migración: create_loans_table

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
```

```
{
    public function up(): void
    {
        Schema::create('loans', function (Blueprint $table) {
            $table->id();

            $table->foreignId('book_id')
                  ->constrained('books')
                  ->restrictOnDelete();

            $table->foreignId('member_id')
                  ->constrained('members')
                  ->restrictOnDelete();

            $table->foreignId('loaned_by')
                  ->nullable()
                  ->constrained('users')
                  ->nullOnDelete();

            $table->date('loan_date');
            $table->date('due_date');
            $table->date('returned_date')->nullable();
            $table->enum('status', ['active', 'returned', 'overdue'])
                  ->default('active');
            $table->text('notes')->nullable();
            $table->timestamps();

            // Índices para consultas frecuentes
            $table->index('status');
            $table->index('due_date');
            $table->index(['member_id', 'status']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('loans');
    }
};
```

Advertencia: RESTRICT on Delete

Las claves foráneas book_id y member_id en la tabla loans utilizan restrictOnDelete(). Esto significa que PostgreSQL impedirá eliminar un libro o un miembro si tienen préstamos asociados. Esta restricción protege la integridad de los datos: no se puede borrar un libro que está prestado ni un miembro con préstamos activos.

3.3.9 Ejecutar las Migraciones

Con todas las migraciones definidas, ejecute:

```
php artisan migrate
```

Laravel ejecutará cada migración en orden cronológico (basado en el timestamp del nombre del archivo). Verá una salida similar a:

```
  INFO  Running migrations.
```

```
  0001_01_01_000000_create_users_table ........... DONE
  0001_01_01_000001_create_cache_table ........... DONE
  0001_01_01_000002_create_jobs_table ............ DONE
  2026_04_01_100000_create_authors_table ......... DONE
  2026_04_01_100001_create_categories_table ...... DONE
  2026_04_01_100002_create_books_table ........... DONE
  2026_04_01_100003_create_author_book_table ..... DONE
  2026_04_01_100004_create_members_table ......... DONE
  2026_04_01_100005_create_loans_table ........... DONE
```

3.3.10 Verificar en PostgreSQL

Conectése a PostgreSQL y verifique que las tablas se crearon correctamente:

```
psql -U postgres -d library_db

-- Listar todas las tablas
\dt

-- Ver la estructura de la tabla books
\d books

-- Ver las claves foráneas de loans
\d loans
```

> **Nota: Comandos útiles de migraciones**
> php artisan migrate:rollback — Revierte el último lote de migraciones ejecutadas.
> php artisan migrate:fresh — Elimina TODAS las tablas y re-ejecuta todas las migraciones. Útil en desarrollo, nunca en producción.
> php artisan migrate:status — Muestra el estado de cada migración (ejecutada o pendiente).

3.4 Git y Versionamiento

Inicialice un repositorio Git para versionar el proyecto desde el inicio.

3.4.1 Inicializar el Repositorio

```
git init
```

3.4.2 Verificar .gitignore

Laravel incluye un archivo .gitignore preconfigurado. Verifique que contiene al menos las siguientes entradas:

```
/vendor
/node_modules
.env
.env.backup
.env.production
/storage/*.key
/public/hot
```

```
/public/storage
```

> **Advertencia**
> El archivo .env contiene credenciales sensibles (contraseñas de base de datos, claves API). NUNCA debe versionarse en Git. Verifique que está en .gitignore antes de hacer su primer commit.

3.4.3 Primer Commit

```
git add .
git commit -m "feat: proyecto library-app con migraciones de biblioteca"
```

3.4.4 Crear Tag de Versión

```
git tag -a v1.0 -m "Guía 4: Configuración del proyecto y base de datos"
```

> **Definición: Tags en Git**
> Un tag (etiqueta) marca un punto específico en la historia del repositorio. Se usa para identificar versiones de entrega. El formato v0.1.0 sigue la convención de Versionamiento Semántico (SemVer): MAYOR.MENOR.PARCHE.

3.4.5 Publicar en GitHub

Cree un repositorio nuevo en GitHub (sin README, sin .gitignore, sin licencia) y ejecute:

```
git remote add origin https://github.com/su-usuario/library-app.git
git branch -M main
git push -u origin main
git push --tags
```

Verifique en GitHub que el código y el tag v1.0 aparecen correctamente.

4. Ejercicios Propuestos

Complete los siguientes ejercicios para reforzar los conceptos aprendidos. Ambos ejercicios deben implementarse como migraciones adicionales en el proyecto.

Ejercicio 1: Tabla fines (Multas)

Cree una migración para la tabla **fines** que almacene las multas por devolución tardía de préstamos. La tabla debe contener los siguientes campos:

Campo	Tipo	Restricciones
`id`	bigIncrements	PK
`loan_id`	FK → loans	CASCADE on delete
`amount`	decimal(8,2)	Requerido

Campo	Tipo	Restricciones
`reason`	string(255)	Requerido
`status`	enum	pending, paid. Default: 'pending'
`paid_at`	date	Nullable
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Ejercicio 2: Tabla book_reviews (Reseñas)

Cree una migración para la tabla **book_reviews** que permita a los miembros calificar y comentar los libros. La tabla debe contener:

Campo	Tipo	Restricciones
`id`	bigIncrements	PK
`book_id`	FK → books	CASCADE on delete
`member_id`	FK → members	CASCADE on delete
`rating`	unsignedTinyInteger	Requerido (1-5)
`comment`	text	Nullable
`created_at`	timestamp	Auto
`updated_at`	timestamp	Auto

Restricción adicional: índice único compuesto en [book_id, member_id] para que cada miembro solo pueda dejar una reseña por libro.

> **Pista para los ejercicios**
> Utilice los mismos patrones que vio en las migraciones anteriores. Recuerde generar cada migración con php artisan make:migration, implementar el método up() con las columnas y restricciones, implementar el método down() con dropIfExists, y ejecutar php artisan migrate para aplicar los cambios.

5. Rúbrica de Evaluación

La evaluación se realizará sobre un total de **100 puntos**, distribuidos de la siguiente manera:

Criterio	Puntos	Descripción
Proyecto creado y conectado a PostgreSQL	15	Proyecto funcional, db:show muestra conexión exitosa.
Migraciones: authors, categories, books	20	Campos correctos, tipos de datos apropiados, índices.
Migración pivote: author_book	10	FK con CASCADE, unique constraint en [author_id, book_id].
Migraciones: members y loans	20	FK correctas, enums, defaults, índices compuestos.
FK, indexes, onDelete correctos	15	Comportamiento CASCADE, SET NULL, RESTRICT apropiado.
Repositorio GitHub con tag v0.1.0	10	Repositorio público, commit descriptivo, tag presente.

Criterio	Puntos	Descripción
Ejercicios: fines + book_reviews	10	Tablas con campos, FK y restricciones correctas.

Rango	Calificación
90 – 100 puntos	Excelente
75 – 89 puntos	Bueno
60 – 74 puntos	Suficiente
< 60 puntos	Insuficiente

6. Entregables

Para completar esta guía, entregue los siguientes elementos:

1. **Repositorio en GitHub** con el proyecto library-app completo.
2. **Tag v1.0** visible en el repositorio con todas las migraciones.
3. **Captura de pantalla** del comando php artisan migrate:status mostrando todas las migraciones ejecutadas.
4. **Captura de pantalla** de PostgreSQL (\dt) mostrando las tablas creadas.
5. **Migraciones de los ejercicios** (fines y book_reviews) incluidas en el repositorio.

> **Fecha límite de entrega**
> Consulte la fecha límite en el calendario académico del curso. Entregas tardías tendrán una penalización del 10% por cada día de retraso.